

AdminController Formlar ve modeller

Ne işe yarar? Neden böyle tasarlandı?
Bir input veritabanına nasıl ulaşır?

Ders 02 · Başlangıç seviyesi
Gerçek kod, veri akışları ve cevaplı alıştırımlar.

Voku CMS · 4c8cf8c kod sürümü
AdminController, admin view'ları ve model tasarımı

İçindekiler

- 01 Okumadan yazmaya geçiyoruz
- 02 Sınıf, bağımlılıklar ve yetki
- 03 Route → action → view haritası
- 04 Index: listeyi ekrana taşıma
- 05 Editor: mevcut formu açma
- 06 New: kayıt oluşturmada form hazırlamak
- 07 Edit.cshtml: input nereden değer alıyor?
- 08 Koşullu alanlar ve layout
- 09 POST: route değeri mi, form değeri mi?
- 10 Save: input ve page neden iki nesne?
- 11 Doğrulama: neden üç katman var?
- 12 Alan atamaları ve boş değerler
- 13 INSERT, UPDATE ve slug bağlantıları
- 14 Hata, başarı ve yönlendirme

İçindekiler · devam

15 LoginModel, hash ve cookie

16 Entity, input model ve view model

17 ContentPage alanlarının tasarım amacı

18 Görsel editörün görünmeyen bağlantısı

19 View'da değişiklik yapınca neler gerekir?

20 Bir düzenlemeyi baştan sona izleyelim

21 Kontrol soruları ve mevcut sınırlar

22 Kod referansı · 1–24

23 Kod referansı · 25–48

24 Kod referansı · 49–72

25 Kod referansı · 73–89

Okumadan yazmaya geçiyoruz

SiteController içeriği gösterir; AdminController onu yönetir.

Önceki derste bir URL'nin veritabanı sorgusuna ve view'a nasıl bağlandığını gördük. Bu derste ters yönü de izleyeceğiz: kullanıcı inputa bir değer yazar, form gönderilir, C# nesnesi oluşur, doğrulanır ve veritabanına kaydedilir.

```
GET /admin/posts/ilk-yazim/edit
  → AdminController.Editor → ContentPage → Edit.cshtml
  → Kullanıcı formu değiştirir
POST /admin/posts/ilk-yazim/edit
  → SavePost → Save → doğrulama → SaveChangesAsync
  → Redirect → Edit.cshtml yeniden açılır
GET /blog/ilk-yazim
  → SiteController → güncel kayıt → ziyaretçi view'ı
```

Rehber, projenin 4c8cf8c sürümündeki davranışını anlatır. Uygulama kodu bu ders için değiştirilmez. Kısaltılmış kodlar öğretici örneklerdir; sonda controller'ın gerçek kodu satır numaralarıyla yer alır.

Bileşen	Görevi
AdminController	Ekranı açar, formu alır, doğrular ve kaydeder.
ContentPage	İçerik entity'si; bu projede aynı zamanda edit formunun modeli.
LoginModel	Yalnızca giriş formunun veri modeli.
Edit.cshtml	Entity alanlarından input üretir.
editor.js	Görsel metin alanlarını BodyHtml değerine aktarır.
SitePageViewModel	Ziyaretçi ekranı için entity ve listeleri paketler.

Her bölümde üç soruyu sorun: Bu veri nereden geliyor? Şimdi bellekte mi, veritabanında mı? Sonuç hangi view'da kullanılıyor?

Sınıf, bağımlılıklar ve yetki

Constructor'da neden hem db hem configuration var?

```
[Authorize(Policy = "AdminAccess"), Route("admin")]
public class AdminController(
    SiteDbContext db, IConfiguration configuration) : Controller
```

db veritabanına erişir. configuration JSON ve ortam değişkenleri gibi kaynaklardan ayar okur. İkisi de ASP.NET Core tarafından constructor'a verilir; controller bağlantı veya ayar dosyasını kendi açmaz. Bu bağımlılıkların kayıt/yükleme düzeni Program.cs'de kurulmuştur.

Route("admin") ortak adres önekidir. Bir action'daki HttpGet("posts/new") ile birleşince GET /admin/posts/new olur. Authorize, AdminAccess politikasını ister. Politikanın ayrıntısı Program.cs'de tanımlıdır: giriş kontrolü kapalıysa izin ver, açıksa kimliği doğrulanmış kullanıcı iste.

Ayar / nitelik	Sonuç
Admin:RequireAuthentication=false	Şu anki durum: /admin giriş istemez; düzenleme de anonim yapılabilir.
Admin:RequireAuthentication=true	Oturumsuz kullanıcı admin ekranı için login'e yönlendirilir.
AllowAnonymous	Login action'larına oturum açmadan erişilebilmesini sağlar.
EnableRateLimiting("login")	Login POST için tanımlı IP bazlı hız sınırını uygular.

Authentication "kimlik var mı?", authorization "bu endpoint için izin var mı?" sorusudur. AllowAnonymous bu sayfadaki form doğrulamasını veya CSRF korumasını kapatmaz. Giriş açık/kapalı tercihi ile formun güvenli gönderilmesi ayrı konulardır.

Şu an login ekranı görünmüyor diye giriş kodu silinmiş değil. Ayar false olduğu için Login action'ı /admin'e yönlendiriyor.

Route → action → view haritası

Aynı URL'nin GET ve POST istekleri farklı iş yapar.

HTTP ve adres	Action / yardımcı	Sonuç
GET /admin	Index	Index.cshtml + List<ContentPage>
GET /admin/pages/{slug}/edit	EditPage → Editor	Edit.cshtml + sabit ContentPage
GET /admin/posts/{slug}/edit	EditPost → Editor	Edit.cshtml + detay ContentPage
GET /admin/posts/new	New	Edit.cshtml + yeni ContentPage
POST /admin/pages/{slug}/edit	SavePage → Save	Kayıt veya hatalı Edit formu
POST /admin/posts/{slug}/edit	SavePost → Save	Kayıt veya hatalı Edit formu
POST /admin/posts/new	Create → Save	INSERT veya hatalı Edit formu
GET /admin/login	Login()	Login.cshtml veya redirect
POST /admin/login	Login(LoginModel)	Doğrulama ve oturum
POST /admin/logout	Logout	Oturumu kapatma ve redirect

GET ekranı okumak ve göstermek içindir. POST kullanıcıdan veri alan işleme gider. Aynı URL'ye sahip iki metod, HttpGet/HttpPost nitelikleri sayesinde ayrılır. İki Login metodunun parametrelerinin farklı olması da C# metod aşırı yükleme örneğidir.

Editor ve Save private yardımcı metotlardır; dışarıya ayrı route olarak açılmazlar. EditPage/EditPost aynı düzenleme işini farklı IsDetail değeriyle kullanır. SavePage/SavePost/Create da ortak doğrulama-kayıt işini paylaşır. Amaç aynı kodu üç kez yazıp zamanla farklı davranmasına yol açmamaktır.

View("Edit", page) yeni bir GET isteği başlatmaz; mevcut isteğin yanıtında Edit view'ını çizer. Redirect(...) ise tarayıcıya başka adrese yeni istek yapmasını söyler.

Index: listeyi ekrana taşıma

Veritabanı sıralaması ile view içindeki gruplama arasındaki fark.

```
public async Task<IActionResult> Index() =>
    View(await db.Pages.AsNoTracking()
        .OrderBy(p => p.IsDetail)
        .ThenBy(p => p.Title)
        .ToListAsync());
```

AsNoTracking okuma içindir. OrderBy önce sabit sayfaları, ardından detayları sıralar; ThenBy aynı grup içinde başlığa göre sıralar. ToListAsync sonuçları getirir. View adı verilmediği için action adı Index kullanılır; dosya Views/Admin/Index.cshtml olur.

```
@model List<ContentPage>
@foreach (var detail in new[] { false, true })
{
    foreach (var entry in Model.Where(p => p.IsDetail == detail))
    {
        <a href="@entry.Url">@entry.Title</a>
    }
}
```

Bu Razor örneği sadeleştirilmiştir. Önemli fark: Controller'daki LINQ EF sorgusudur; view'daki Model artık bellekte List<ContentPage> olduğundan Where bellekte gruplar. View'daki her Where yeni SQL göndermez.

Admin listesi Published filtresi kullanmaz. Taslaklar da görünmelidir; aksi halde onları yeniden yayınlamak veya düzeltmek mümkün olmazdı. Ziyaretçi listesi ise yalnızca yayınlanmış kayıtları gösterir.

Düzenleme bağlantısı IsDetail değerinden üretilir: sabit sayfada /admin/pages/about/edit, yazıda /admin/posts/ilk-yazim/edit. Url ise ziyaretçi bağlantısıdır. Yönetim linki ile ziyaretçi linkinin aynı olmaması gerekir.

View model alanı ile HTML etiketi aynı şey değildir: Title veridir; onu h1, td veya a içinde göstermek view'ın kararıdır. Görünümü değiştirmek her zaman veritabanı değişikliği gerektirmez.

Editor: mevcut formu açma

GET sırasında BodyHtml değişiyor; bu hemen veritabanına yazılır mı?

```
var page = await db.Pages.AsNoTracking()
    .SingleOrDefaultAsync(p => p.Slug == slug && p.IsDetail == detail);
if (page == null) return NotFound();
page.BodyHtml = page.IsDetail ? BlogMarkup.ArticleBody(page.BodyHtml)
    : page.Slug is "home" or "blog"
    ? BlogMarkup.ListingTemplate(page.BodyHtml) : page.BodyHtml;
return View("Edit", page);
```

1. URL'deki slug ve route'un belirlediği detail ile kayıt aranır.
2. Kayıt yoksa NotFound ile işlem sonlanır.
3. Blog detayında eski şablondan yalnızca makale gövdesi çıkarılır.
4. Ana sayfa/blogda örnek kart alanı BLOG_LIST işaretine uyarlanır.
5. Hazırlanan ContentPage, Edit.cshtml'e gönderilir.

Burada page.BodyHtml ataması yalnızca bellekteki nesneyi değiştirir. Sorgu AsNoTracking ile çalışmıştır ve SaveChangesAsync de çağırılmamıştır. Formu açıp kapatmak veritabanını güncellemez. Ancak bu hazırlanmış değer formda görünür; kullanıcı kaydederse POST yolundan kalıcı olur.

Neden Published kontrolü yok? Taslak yazının editörü de açılmalıdır. Neden ArticleBody uygulanıyor? Yönetici makale yazarken header/footer ve eski örnek yorumlarla uğraşmasın; dinamik başlıklar ayrı alanlardan yönetilsin.

Bir nesnenin özelliğine değer atamak her zaman SQL UPDATE değildir. Kalıcılık için doğru context takibi ve kaydetme adımı gerekir. Bu ayrım GET ile POST arasındaki en önemli farklardan biridir.

New: kayıt oluşturmada form hazırlamak

Yeni nesne ile yeni veritabanı satırı aynı şey değildir.

```
public IActionResult New() => View("Edit", new ContentPage {
    IsDetail = true, Published = false, Category = "Genel",
    CoverImageUrl = "/images/blog-post-bg.jpg",
    BodyHtml = "<section><p>Yazınızı buraya ekleyin.</p></section>"
});
```

HTML örneği kısaltılmıştır. new ContentPage bellekte bir nesne oluşturur. Add ve SaveChanges olmadığı için tabloya satır eklenmez. Id varsayılan int değeri 0 olduğu için Edit view'ı "Yeni detay" başlığını seçer; mevcut kayıta ise Title gösterir.

Başlangıç alanı	Neden böyle?
IsDetail=true	Bu ekran blog yazısı oluşturuyor.
Published=false	Tamamlanmamış yazı kaydedilirse hemen ziyaretçiye açılmasın.
Category="Genel"	İlk form için düzenlenebilir başlangıç değeri.
CoverImageUrl	Başlangıç görsel yolu; dosya yüklemes.
BodyHtml	Görsel editörün düzenleyebileceği başlangıç paragrafı.

Neden ayrı New.cshtml yok? Yeni yazı ve mevcut yazı aynı alanları düzenliyor. Tek Edit view'ı ile alan görünümü ve hata gösterimi ortak tutuluyor. Formun gönderildiği URL farklı olduğu için POST yeni kayıta Create action'ına, mevcut kayıta SavePost action'ına gidiyor.

ContentPage sınıfında Published varsayılanı true olmasına rağmen New burada açıkça false veriyor. Nesne oluşturulurken yapılan bu atama başlangıç değerini değiştirir. Böylece aynı model farklı kullanımda farklı başlangıç davranışı gösterebilir.

Tarayıcıda formu açmak, sayfayı yenilemek veya vazgeçmek bir yazı oluşturmamalıdır. Yeni kayıt ancak başarılı POST akışındaki Add + SaveChangesAsync ile oluşur.

Edit.cshtml: input nereden değer alıyor?

asp-for, name ve model binding ilişkisi.

```
@model ContentPage
<form method="post">
    @Html.AntiForgeryToken()
    <div asp-validation-summary="All"></div>
    <label asp-for="Title">Başlık</label>
    <input asp-for="Title" maxlength="160" required>
    <textarea asp-for="Description"></textarea>
    <input asp-for="Published">
    <textarea asp-for="BodyHtml"></textarea>
    <button>Değişiklikleri kaydet</button>
</form>
```

Bu, mevcut formun sadeleştirilmiş halidir. @model ContentPage view'ın beklediği türü belirtir. Model, controller'dan gelen nesnedir. asp-for="Title" modeldeki Title özelliğine bağlanır; name="Title", id ve mevcut değer gibi HTML özelliklerini üretir. Model binding POST'taki Title değerini input.Title özelliğine taşır.

Formda action yok; tarayıcı POST'u mevcut URL'ye gönderir. Label'daki asp-for etiketi ilgili inputla ilişkilendirir. HTML placeholder örnek ipucudur, veritabanına kaydedilecek gerçek değer değildir.

View'daki ifade	Gönderim / anlam
input asp-for="Title"	Title adlı form alanı → input.Title
textarea asp-for="BodyHtml"	HTML metni → input.BodyHtml
input asp-for="Published"	Bool checkbox; false gönderimi için yardımcı hidden alan da üretilebilir.
Html.AntiForgeryToken()	CSRF kontrolüne ait gizli token; entity alanı değil.
asp-validation-summary="All"	ModelState içindeki hataları kullanıcıya gösterir.

Sadece name="BaskaAd" yaparsanız Title ile otomatik eşleşme bozulur. Etiket metnini "Yazı başlığı" yapmak ise binding'i değiştirmez. Kullanıcının gördüğü label ile sunucunun bağladığı alan adı ayrı şeylerdir.

Koşullu alanlar ve layout

Neden bazı inputlar yalnızca blogda görünüyor?

Edit view'ında Model.IsDetail kontrolü kapak, kategori, yazar ve yazı tarihini yalnızca blog detayında gösterir. Sabit sayfada Slug readonly olur. Ana sayfa ve blog sabit sayfasında ise kartların otomatik oluştuğunu anlatan bilgi gösterilir.

```
@if (Model.IsDetail)
{
    <input asp-for="Author">
    <input asp-for="PublishedAtUtc" type="datetime-local"
        asp-format="{0:yyyy-MM-ddTHH:mm}">
}
```

Tarih formatı inputun beklediği biçimi sağlar. datetime-local saat dilimi taşımaz; projede alan açıkça UTC diye etiketlenmiştir. Controller'da SpecifyKind(...Utc) sayısal saati dönüştürmez, UTC olarak işaretler. Yerel saati otomatik UTC'ye çevirdiğini düşünmeyin.

1. Views/_ViewImports.cshtml modellerin namespace'ini ve MVC Tag Helper'larını etkinleştirir.
2. Kök _ViewStart normal _Layout seçer; Admin/_ViewStart bunu _AdminLayout ile değiştirir.
3. _AdminLayout header, admin.css, çıkış formu ve bildirim alanını içerir.
4. RenderBody(), Index/Edit/Login view'ının HTML'sini ortak kabuğa yerleştirir.

Admin view'ları kendi başlarına tam html/head/body belgeleri değildir. Ortak kabuk layouttan gelir. Ziyaretçi Page.cshtml ise Layout=null kullanır ve tam belgeyi kendisi kurar. Aynı yaklaşımı her ekrana zorla uygulamak yerine mevcut yapının amacını görmek önemlidir.

Readonly görünüm için yararlıdır ama sunucu güvenliği değildir. Kullanıcı farklı POST gönderebilir. Save metodunda sabit slug'ın değişmediği ayrıca kontrol edilir.

POST: route değeri mi, form değeri mi?

Slug değiştirirken iki farklı slug aynı anda vardır.

```
public Task<IActionResult> SavePost(
    [FromRoute] string slug, ContentPage input)
    => Save(slug, true, input);

public Task<IActionResult> Create(ContentPage input)
    => Save(null, true, input);
```

Örnek: URL /admin/posts/eski-ad/edit, formdaki Slug ise yeni-ad. slug parametresi mevcut kaydı bulmak için eski-ad olmalıdır. input.Slug yeni kaydedilecek değerdir. FromRoute özellikle URL'deki değer kullanıldığını belirtir; formdaki aynı isimli alanla karışmasını engeller.

Değer	Kaynağı	Amacı
oldSlug	URL; yenide null	Mevcut kaydı bulmak / yeni kayıt ayrımı
input.Slug	POST formu	Kaydedilecek yeni adres
detail	SavePost/SavePage action'ı	Yazı mı sabit sayfa mı?
input	Model binding	Gönderilen alanları taşıyan geçici nesne

SavePage false, SavePost true, Create null/true gönderir. Kullanıcının formda IsDetail=true göndermesi sabit sayfa endpointini blog endpointine dönüştürmez; Save bu değeri route'a göre yeniden atar.

Task doğrudan döndürüldüğünden bu kısa action'larda async/await yoktur. Asenkron işlemler ortak Save içinde beklenir. Metotların küçük olması işin önemsiz olduğunu değil, işi tek ortak yere aktardığını gösterir.

Bu action'lar yalnızca POST kabul eder. Aynı URL'ye GET ile gitmek düzenleme formunu açan action'ı seçer; kayıt işlemini çalıştırmaz.

Save: input ve page neden iki nesne?

İstemci verisini gerçek entity'ye kontrollü aktarmak.

```
input.IsDetail = detail;
var page = oldSlug == null
    ? new ContentPage { IsDetail = true }
    : await db.Pages.SingleOrDefaultAsync(p =>
        p.Slug == oldSlug && p.IsDetail == detail);
if (page == null) return NotFound();
input.Id = page.Id;
```

input, formdan gelen nesnedir. page, veritabanından yüklenen takipli kayıt veya yeni oluşturulan gerçek entity'dir. Burada AsNoTracking kullanılmaz; mevcut page üzerindeki değişikliklerin SaveChangesAsync tarafından bulunması istenir.

oldSlug null ise yeni kayıt oluşturulur. Null değilse doğru tür ve eski slug ile kayıt aranır. Başkası kaydı kaldırmışsa veya URL yanlışsa NotFound döner. input.Id gerçek page.Id ile değiştirilir; gelen Id değerine güvenilmez. Bu Id özellikle hata halinde edit view'ının mevcut/yeni kayıt ayrımı için de önemlidir.

```
page.Title = input.Title;
page.Slug = input.Slug;
page.Description = input.Description ?? "";
```

Neden doğrudan db.Update(input) kullanılmıyor? İstemci tarafından gönderilebilecek tüm entity özelliklerinin otomatik kayda aktarılması yerine hangi alanların değişeceği açıkça seçiliyor. IsDetail, Id ve UpdatedUtc sunucunun kontrolünde tutuluyor. Bu yaklaşım ayrı input modeli bulunmasa bile kontrolü artırıyor.

Bu işlem yeni HTTP isteğinde gerçekleşir. GET'te oluşturulan page nesnesi POST boyunca bellekte bekletilmiyor. POST kendi inputunu oluşturuyor ve gerçek kaydı yeniden veritabanından yüklüyor.

Doğrulama: neden üç katman var?

HTML, ModelState ve veritabanı birbirinin yerine geçmez.

```
if (!detail && oldSlug != input.Slug)
    ModelState.AddModelError("Slug", "Sabit sayfaların route'u değiştirilemez.");
if (await db.Pages.AnyAsync(p => p.IsDetail == detail
    && p.Slug == input.Slug && p.Id != page.Id))
    ModelState.AddModelError("Slug", "Bu slug zaten kullanılıyor.");
if (!ModelState.IsValid) return View("Edit", input);
```

AnyAsync yalnızca böyle bir kayıt var mı diye sorar; liste getirmez. `p.Id != page.Id` koşulu, mevcut yazının kendi slug'ıyla çakışmış sayılmamasını sağlar. Başka kayıta aynı tür/slug varsa hata eklenir.

Katman	Örnek	Neden var?
Tarayıcı	required, maxlength, readonly	Hızlı kullanıcı geri bildirimi; tek güvence değil.
Model / controller	Required, regex, ModelState	Tarayıcı dışından gelen isteği de denetler.
Veritabanı	UNIQUE(IsDetail,Slug)	Eşzamanlı iki isteğin aynı slug'ı oluşturmaya son engel.

ModelState model binding sırasında oluşan dönüştürme hatalarını ve doğrulama hatalarını tutar. Örneğin tarih alanı tarihe çevrilemiyorsa veya slug büyük harf içeriyorsa geçersiz olabilir. Hata halinde `View("Edit", input)` kullanıcının girdilerini koruyarak aynı formu gösterir; redirect yapılmaz.

`AddModelError("Slug", ...)` belirli alana; `AddModelError("", ...)` genel forma hata ekler. `asp-validation-summary="All"` ikisini de gösterebilir. ModelState'in sakladığı gönderilmiş değerler, hata formunda Tag Helper'ların yeniden çizdiği değerlere öncelik verebilir; model özelliğini değiştirmek her zaman ekrandaki hatalı girdiyi değiştirmez.

HTTP 200, kaydın başarılı olduğunu tek başına göstermez. Geçersiz form da hata mesajlarıyla 200 dönebilir. Başarılı Save akışı burada redirect üretir.

Alan atamaları ve boş değerler

Null, boş string ve tarih kararlarının amacı.

```
page.CoverImageUrl = string.IsNullOrEmpty(input.CoverImageUrl)
    ? null : input.CoverImageUrl.Trim();
page.Category = string.IsNullOrEmpty(input.Category)
    ? null : input.Category.Trim();
page.Author = string.IsNullOrEmpty(input.Author)
    ? null : input.Author.Trim();
page.PublishedAtUtc = DateTime.SpecifyKind(input.PublishedAtUtc, DateTimeKind.Utc);
```

Bu alanlar yalnızca detail=true için atanır. IsNullOrEmpty null, boş metin veya sadece boşluk girilmiş olmasını tek yerde kontrol eder. Dolu değerler Trim ile baş/son boşluklarından temizlenir. Böylece " Gezi " gibi istemeden farklı görünen değerler azaltılır.

Description farklı davranır: input.Description ?? "". DbContext bu sütunu zorunlu tanımladığı için null yerine boş string saklanır. Modelin C# tarafında nullable olması ile veritabanının null kabul etmesi ayrı ayarlardır.

BodyHtml için blogda ArticleBody, ana sayfa/blog listesinde ListingTemplate uygulanır. Bu, bilinen Voku şablon yapısını düzenler; genel bir HTML sanitizier değildir. Published formdan alınır, UpdatedUtc ise DateTime.UtcNow ile sunucuda yenilenir.

Tarih	Amacı
PublishedAtUtc	Yönetilebilir gösterim ve sıralama tarihi. İleri tarih otomatik zamanlama oluşturmaz.
UpdatedUtc	Son kaydın sunucu zamanı. Kullanıcı formundan alınmaz.

Görsel alanının regex'i adres biçimini sınırlar; dosyanın gerçekten var olduğunu veya yüklenebildiğini kanıtlamaz. Kapak yolu yazmak bir dosya upload işlemi değildir.

INSERT, UPDATE ve slug bağlantıları

Add ile SaveChangesAsync neden ayrı?

```
if (oldSlug == null) db.Pages.Add(page);
else if (detail && oldSlug != page.Slug)
{
    foreach (var linked in await db.Pages
        .Where(p => p.BodyHtml.Contains(oldUrl)).ToListAsync())
    {
        linked.BodyHtml = linked.BodyHtml
            .Replace("\" + oldUrl + "\"", "\" + page.Url + "\"")
            .Replace("'" + oldUrl + "'", "'" + page.Url + "'");
    }
    await db.SaveChangesAsync();
}
```

Add, yeni entity'nin INSERT ile kaydedileceğini context'e bildirir. Mevcut entity zaten takipli sorguyla alındığından özellik atamaları değişiklik olarak izlenir; ayrıca Add yapılmaz. SaveChangesAsync bekleyen değişiklikleri veritabanına gönderir. Önceki atamalar yalnızca bellekteydi.

Slug değişiyorsa önceki URL oldUrl içinde korunmuştur. Eski adresi içeren BodyHtml kayıtları aranır; tek/çift tırnak içinde tam eşleşen bağlantılar yeni adresle değiştirilir. Aynı kaydetme çağrısında bu takipli kayıtlar da güncellenir.

Dinamik kartların bağlantısı post.Url'den hesaplandığı için HTML Replace işlemine ihtiyaç duymaz. Bu döngü elle yazılmış mevcut HTML bağlantıları içindir. Harici sitelerdeki linkleri, farklı query/fragment biçimlerini veya her tür HTML yazımını dönüştüren tam bir parser değildir.

Eski slug için redirect geçmişi tutulmaz. /blog/eski-ad artık 404 olabilir. Slug değişikliğiyle mevcut iç bağlantıyı değiştirmek, dışarıdan gelen eski URL'yi kalıcı yönlendirmekle aynı özellik değildir.

Hata, başarı ve yönlendirme

Neden başarılı POST sonrası tekrar GET yapılıyor?

```
try { await db.SaveChangesAsync(); }
catch (DbUpdateException)
{
    ModelState.AddModelError("", "Kayıt tamamlanamadı. Slug başka bir kayıta kullanılıyor olabilir.");
    return View("Edit", input);
}
TempData["Success"] = "İçerik kaydedildi.";
return Redirect(detail
    ? $"/admin/posts/{page.Slug}/edit"
    : $"/admin/pages/{page.Slug}/edit");
```

Ön kontrol geçse bile başka istek aynı anda aynı slug'ı kaydedebilir. Veritabanı benzersiz indeksi bunu engeller; SaveChangesAsync hata verebilir. Catch, DbUpdateException yakalayıp formu mesajla döndürür. Ancak bu exception yalnızca slug hatası değildir; mesaj olası nedeni belirtir, kesin teşhis yerine geçmez.

Başarılı akış POST → 302 redirect → GET biçimindedir. Bu düzen, kullanıcı başarı ekranını yenilediğinde aynı formu tekrar gönderme riskini azaltır. Redirect aynı kaydın yeni slug adresine yapıldığı için editörde güncel route görünür.

TempData mesajı yönlendirmeden sonraki isteğe taşır. _AdminLayout, TempData["Success"] string ise role="status" olan bildirim paragrafında gösterir. Sıradan yerel değişken redirect sonrası yeni isteğe aktarılmaz.

Return View ile Redirect farkını pratikte görün: geçersiz slug gönderince aynı form hata gösterir; geçerli kayıta Network sekmesinde önce POST, ardından GET görünür.

LoginModel, hash ve cookie

Giriş kontrolü tekrar açılırsa bu kod nasıl çalışacak?

1. GET Login(), ayar true ise new LoginModel ile Login.cshtml açar; false ise Index'e yönlendirir.
2. POST Login(input), önce ayarı ve ModelState'i kontrol eder.
3. AdminUsers içinde Username eşleşen kullanıcıyı arar.
4. Kayıt yoksa veya PasswordHasher doğrulaması başarısızsa genel hata mesajı verir.
5. Başarıda kullanıcı adı claim'i olan kimlik oluşturur; SignInAsync authentication cookie'sini üretir.
6. Index'e yönlendirir. Sonraki isteklerde middleware cookie'den kullanıcı kimliğini okur.
7. POST Logout(), SignOutAsync ile tarayıcı oturumunu kapatır; login'e yönlendirir.

LoginModel.Password düz şifre girişidir; AdminUser.PasswordHash saklanan doğrulama değeridir. Şifreyi bulmak için hash'i geri çözme akışı yoktur. DataType.Password inputun görünümünü belirtir; kendisi hashleme yapmaz. Hash kontrolünü PasswordHasher yapar.

ClaimsPrincipal kullanıcının kimlik bilgisini taşır. ClaimTypes.Name bu kodda kullanıcı adıdır. Tam ASP.NET Core Identity kullanıcı/rol sistemi kurulmuş değildir; yalnızca PasswordHasher ve cookie authentication bileşenleri kullanılır. Rol tabanlı yetki ayrımı yoktur.

Cookie adının Voku.Admin olması ve dört saatlik bilet süresi Program.cs'de ayarlanır. Login POST için IP bazlı dakikada 10 istek limiti tanımlanır. Çıkışın POST olması, durum değiştiren bir işlem olmasıyla uyumludur; layout formu antiforgery token gönderir.

Şu an ayar false olduğu için hesap şifresi bu ekranların ön koşulu değil. Buna rağmen login akışını öğrenmek, ayar yeniden true yapıldığında hangi kodun devreye gireceğini anlamanızı sağlar.

Entity, input model ve view model

Neden her model için bir tablo yok?

Tür	Nerede kullanılıyor?	Tablo / amaç
ContentPage	EF, Admin/Edit, Save parametresi	Pages entity'si; mevcut projede üç görev bir sınıfta.
AdminUser	Girişte kullanıcı sorgusu	AdminUsers entity'si; hash saklar.
LoginModel	Login.cshtml ve POST Login	Form girdisi; DbSet olmadığı için ayrı tablo yok.
SitePageViewModel	SiteController → Site/Page	Ekran paketi; entity + listeler + sayfalama.

Entity verinin kalıcı biçimini, input modeli kullanıcının gönderebileceği alanları, view model ekranın ihtiyaç duyduğu veri paketini anlatır. İsimde Model geçmesi tek başına “veritabanı tablosu” demek değildir. EF eşlemesi DbContext/model yapılandırmasına bağlıdır.

Bu küçük projede ContentPage hem entity hem edit modeli seçilerek alanların tekrar yazılması azaltılmış. Bedeli, kalıcılık ayrıntılarının forma da görünür olmasıdır. Bu yüzden Save, Id ve IsDetail gibi alanları istemciden alıp körlemesine kaydetmez; izin verilen alanları tek tek kopyalar.

```
// Gelecekte değerlendirilebilecek tasarım; mevcut kod değil:
public class EditPostInput
{
    public string Title { get; set; } = "";
    public string Slug { get; set; } = "";
    public string BodyHtml { get; set; } = "";
}
// POST EditPostInput alır; gerçek entity ayrıca yüklenir.
```

Ayrı input modeli özellikle daha fazla yetki, çoklu form veya karmaşık veri ilişkisi olduğunda hangi alanların değiştirilebileceğini açıklar. SitePageViewModel ise birkaç farklı sorgu sonucunu aynı view'a taşımak için zaten ayrılmıştır.

DbContext'te Pages ve AdminUsers vardır. Yazar adı Author bir string'dir; AdminUser'a foreign key bağlantısı yoktur. Category de ayrı tablo ilişkisi değil string alandır. Bu yüzden mevcut sistemi ilişkili kullanıcı/kategori sistemi gibi düşünmeyin.

ContentPage alanlarının tasarım amacı

Hangi alan kullanıcıdan gelir, hangisi hesaplanır?

Alan	Kaynak / kural	Niçin?
Id	Veritabanı anahtarı; form belirlemez	Kaydı güvenilir biçimde ayırır.
Title	Zorunlu, en çok 160	Liste ve detay ana başlığı.
Slug	Zorunlu, 120, küçük harf/rakam/tire	Okunabilir benzersiz adres.
IsDetail	Endpoint belirler	Sabit sayfa ile yazıyı aynı tabloda ayırır.
BodyHtml	Zorunlu HTML gövdesi	Zengin içerik ve Voku düzenini korur.
Description	300; kayıt sırasında null yerine boş string	Özet ve SEO açıklaması.
CoverImageUrl	500; yerel yol veya HTTPS biçimi	Kart/detay ortak görseli.
Category / Author	80 / 100; boşsa null	Filtre ve yazar metni.
PublishedAtUtc	Formun UTC tarihi	Sıralama ve görünür tarih.
Published	Form bool değeri	Ziyaretçiye açık mı?
UpdatedUtc	Sunucuda DateTime.UtcNow	Son kayıt zamanı.
Url	Slug ve IsDetail'den hesaplanır	Aynı adres bilgisini iki kez saklamaz.

Required, StringLength ve RegularExpression attribute'ları model doğrulamasına katılır. HTML'de required/maxlength yazılması bu sunucu kurallarının yerine geçmez. StringLength'in SQLite'da tek başına katı uzunluk sınırı oluşturduğunu varsaymayın; bu projede MVC doğrulaması önemlidir.

DbContext Url alanını Ignore eder: sütun üretilmez. IsDetail/Slug birleşik unique indeksi aynı türde çakışmayı önler. Description için IsRequired, nullable C# özelliğine rağmen veritabanında NOT NULL istediğimizi belirtir. Bu yüzden Save boş açıklamayı dönüştürür.

Bir alanın ekranda gösterilmesi, onun veritabanında ayrı sütun olmasını gerektirmez. Url ve sayfalama TotalPages bunun iki örneğidir.

Görsel editörün görünmeyen bağlantısı

Birçok kutu var ama neden yalnızca BodyHtml kaydediliyor?

```
const doc = new DOMParser().parseFromString(source.value, "text/html");
const sync = () => { source.value = doc.body.innerHTML; };
input.addEventListener("input", () => {
  update(input.value);
  sync();
});
```

source, id=BodyHtml olan textarea'dır. editor.js bu metni geçici DOM belgesine çevirir. Metin elemanları için input/textarea, bağlantılar için href, görseller için src/alt alanları üretir. Kullanıcı alanı değiştirdikçe geçici belge ve asıl textarea güncellenir.

1. GET Editor, BodyHtml'yi Edit view'ına gönderir.
2. asp-for textarea'sı bu değeri taşır.
3. JavaScript görsel alanları oluşturur.
4. Kullanıcı paragrafı değiştirir; sync() tüm body HTML'sini textarea'ya yazar.
5. Normal form POST'u BodyHtml alanını gönderir.
6. Save input.BodyHtml değerini gerçek entity'ye aktarır ve kaydeder.

Görsel alanların her biri için ayrı veritabanı sütunu veya endpoint yoktur. İç içe HTML elemanları bulunan metinler görsel editörde tek kutu olarak açılmaz; yapı bozulmasın diye gelişmiş HTML alanı kullanılabilir. Görsel yolu değiştirmek dosya yüklemek değildir.

cms.js ziyaretçi form gönderimini durdurur; admin layoutu onu yüklemeyiz. Admin/Edit yalnızca editor.js ekler ve normal POST çalışır. "İletişim formu gönderilmiyor ama admin kaydediyor" farkının kaynağı hangi scriptin hangi view'da yüklendiğidir.

Razor'daki textarea, BodyHtml metnini editörde kodlanmış biçimde gösterir. Ziyaretçi tarafında Html.Raw ile markup olarak çizilir. HTML'i düzenlemek ile HTML'i çalışır görünüm olarak sunmak ayrı aşamalardır.

View'da değişiklik yapınca neler gerekir?

Sadece görünüm değişikliği ile yeni veri alanını ayıralım.

İstek	Değişiklik yeri	Migration?
Buton metni veya renk değişsin	Edit.cshtml / admin.css	Hayır.
Başlık etiketi farklı yazsın	label metni; asp-for aynı kalabilir	Hayır.
Var olan Author kartta gösterilsin	_BlogListing.cshtml	Hayır; veri zaten geliyor.
Yeni ReadingMinutes alanı saklansın	Entity + migration + form + Save + ziyaretçi view	Evet.
Kartlarda Title yerine Heading adı kullanılsın	Ayrı view model / projection	Veritabanı adı değişmiyorsa hayır.

Öğretici alıştırma, mevcut özellik değil: ReadingMinutes eklemek için int? alanını ContentPage'de tanımlayın; örneğin Range ile makul değerleri doğrulayın. Yeni migration üretilip uygulanacak şemayı inceleyin. Edit'e asp-for input, Save'e açık atama, kart/detaya gösterim ekleyin. Son olarak POST ile kaydedilip GET'te görüldüğünü test edin.

```
// Alıştırma örneği; uygulamaya eklenmedi:
[Range(1, 240)]
public int? ReadingMinutes { get; set; }

// Edit.cshtml:
<input asp-for="ReadingMinutes">

// Save içinde:
page.ReadingMinutes = input.ReadingMinutes;
```

Neden bütün adımlar gerekiyor? Input olmadan kullanıcı değer giremez. Save ataması olmadan gönderilen değer kalıcı entity'ye taşınmaz. Migration olmadan eski veritabanında sütun olmaz. Ziyaretçi view'ı kullanmazsa kaydedilen veri görünmez.

Bu çalışma bir eğitim belgesidir; örnek alan eklenmedi. Amaç, tek bir input eklemenin neden her zaman özelliği tamamlamadığını göstermek.

Bir düzenlemeyi baştan sona izleyelim

Yazar adını değiştirip ziyaretçi kartında görelim.

1. GET /admin/posts/ilk-yazim/edit → EditPost → Editor.
2. Editor ContentPage'yi alır; Edit view'ında input asp-for="Author" mevcut yazar adıyla dolar.
3. Kullanıcı Author alanına "Ayşe" yazar. Henüz SQLite değişmez.
4. Form POST olur; token doğrulanır, model binding input.Author="Ayşe" yapar.
5. SavePost eski slug'ı route'tan alır ve Save'e aktarır.
6. Save gerçek page entity'sini takipli yükler, doğrulamayı kontrol eder.
7. Trim edilmiş Author değeri page.Author'a atanır; UpdatedUtc güncellenir.
8. SaveChangesAsync veritabanını yazar. TempData mesajı konur, edit URL'sine yönlendirilir.
9. Yeni GET formda "Ayşe"yi gösterir.
10. Ziyaretçi /blog açınca SiteController Posts listesinde güncel entity'yi getirir; _BlogListing @post.Author ile "Ayşe"yi gösterir.

Hata varyasyonu: Aynı anda geçersiz slug gönderin. ModelState başarısız olduğundan SaveChanges'e ulaşmaz. Formda girdiğiniz yazar adı görünebilir ama veritabanındaki yazar değişmemiş olabilir. Formda bir değerin görünmesi tek başına kaydedildiğinin kanıtı değildir.

Taslak varyasyonu: Published kutusunu kaldırıp kaydedin. Admin listesi kaydı göstermeye devam eder; ziyaretçi listesi ve detay sorgusu Published şartıyla dışlar. Veritabanından silinmemiştir, yayın durumu değişmiştir.

Breakpoint noktaları: Save başlangıcı → ModelState kontrolü → page.Author ataması → SaveChangesAsync sonrası. input ile page değerlerini ayrı ayrı izleyin. Tarayıcı Network'te POST ve takip eden GET'i görün.

Kontrol soruları ve mevcut sınırlar

Neyi öğrendik, bu uygulamada henüz ne yok?

Soru	Cevap
Formun name alanı neden önemli?	Model binding hangi özelliğe atama yapacağını onunla eşleştirir.
GET formunu açmak niçin kayıt oluşturmaz?	New/Editor Add + SaveChanges çağırılmaz.
AsNoTracking neden Editor'da var, Save'de yok?	Editor gösterim için okur; Save mevcut entity değişikliklerini takip ettirir.
input.Slug ile oldSlug neden farklı olabilir?	Form yeni adresi, route mevcut kaydın eski adresini taşır.
LoginModel için migration gerekir mi?	EF entity'si olarak eşlenmiyor; yalnızca form modeli.
Title değişince başka view'a veri nasıl gider?	Sonraki SiteController sorgusu güncel entity'yi getirir.
Category bir foreign key mi?	Hayır; bu sürümde string alan.
CSRF token giriş kontrolü mü?	Hayır; yetki politikası ayrı, form isteği doğrulaması ayrı.

Mevcut sistemde ayrı edit input modeli, eşzamanlı düzenleme çatışması tespiti, sürüm geçmişi, silme endpointi, slug yönlendirme tablosu ve yönetici şifre sıfırlama komutu yoktur. Her eksik özellik bu dersin hatası değildir; uygulamanın mevcut kapsamıdır.

HTML gövdesi güvenilir yönetici düzenlemesi için tasarlanmıştır. BlogMarkup bir sanitizier değildir. ModelState bazı biçimleri doğrular; her iş kuralını kendiliğinden keşfetmez. Örneğin kapak adresinin erişilebilir olması ayrı bir kontroldür.

Son alıştırma: "Edit.cshtml'de bir değeri değiştirdiğinde hangi action çalışır, hangi modele bağlanır, hangi entity'ye kopyalanır, hangi satırda kalıcı olur ve ziyaretçi view'ında nasıl görünür?" sorusunu Title ve Author için ayrı cevaplayın.

Kod referansı · 1–24

Mevcut AdminController.cs; uzun satırlar yalnızca sayfaya sığmak için sarılır.

```
01 using System.Security.Claims;
02 using Microsoft.AspNetCore.Authentication;
03 using Microsoft.AspNetCore.Authentication.Cookies;
04 using Microsoft.AspNetCore.Authorization;
05 using Microsoft.AspNetCore.Identity;
06 using Microsoft.AspNetCore.Mvc;
07 using Microsoft.AspNetCore.RateLimiting;
08 using Microsoft.EntityFrameworkCore;
09 using Voku.Web.Data;
10 using Voku.Web.Models;
11 namespace Voku.Web.Controllers;
12 [Authorize(Policy = "AdminAccess"), Route("admin")]
13 public class AdminController(SiteDbContext db, IConfiguration configuration) : Controller
14 {
15     [AllowAnonymous, HttpGet("login")]
16     public IActionResult Login() => configuration.GetValue("Admin:RequireAuthentication", true)
17         ? View(new LoginModel()) : RedirectToAction(nameof(Index));
18     [AllowAnonymous, HttpPost("login"), EnableRateLimiting("login")]
19     public async Task<IActionResult> Login(LoginModel input)
20     {
21         if (!configuration.GetValue("Admin:RequireAuthentication", true)) return
RedirectToAction(nameof(Index));
22         if (!ModelState.IsValid) return View(input);
23         var user = await db.AdminUsers.SingleOrDefaultAsync(u => u.Username == input.Username);
24         if (user == null || new PasswordHasher<AdminUser>().VerifyHashedPassword(user, user.PasswordHash,
input.Password) == PasswordVerificationResult.Failed)
```

Kod referansı · 25–48

Mevcut AdminController.cs; uzun satırlar yalnızca sayfaya sığmak için sarılır.

```

25         { ModelState.AddModelError("", "Kullanıcı adı veya şifre hatalı."); return View(input); }
26         await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, new
ClaimsPrincipal(new ClaimsIdentity(new[] { new Claim(ClaimTypes.Name, user.Username) },
CookieAuthenticationDefaults.AuthenticationScheme)));
27         return RedirectToAction(nameof(Index));
28     }
29     [HttpPost("logout")]
30     public async Task<IActionResult> Logout() { await HttpContext.SignOutAsync(); return
RedirectToAction(nameof(Login)); }
31     [HttpGet("")]
32     public async Task<IActionResult> Index() => View(await db.Pages.AsNoTracking().OrderBy(p =>
p.IsDetail).ThenBy(p => p.Title).ToListAsync());
33     [HttpGet("pages/{slug}/edit")]
34     public Task<IActionResult> EditPage(string slug) => Editor(slug, false);
35     [HttpGet("posts/{slug}/edit")]
36     public Task<IActionResult> EditPost(string slug) => Editor(slug, true);
37     private async Task<IActionResult> Editor(string slug, bool detail)
38     {
39         var page = await db.Pages.AsNoTracking().SingleOrDefaultAsync(p => p.Slug == slug && p.IsDetail ==
detail);
40         if (page == null) return NotFound();
41         page.BodyHtml = page.IsDetail ? BlogMarkup.ArticleBody(page.BodyHtml)
42             : page.Slug is "home" or "blog" ? BlogMarkup.ListingTemplate(page.BodyHtml) : page.BodyHtml;
43         return View("Edit", page);
44     }
45     [HttpGet("posts/new")]
46     public IActionResult New() => View("Edit", new ContentPage {
47         IsDetail = true, Published = false, Category = "Genel",
48         CoverImageUrl = "/images/blog-post-bg.jpg",

```

Kod referansı · 49–72

Mevcut AdminController.cs; uzun satırlar yalnızca sayfaya sığmak için sarılır.

```

49         BodyHtml = "<section class=\"container margin-top-50 margin-bottom-50\"><p>Yazınızı buraya ekleyin.
</p></section>"
50     });
51     [HttpPost("pages/{slug}/edit")]
52     public Task<IActionResult> SavePage([FromRoute] string slug, ContentPage input) => Save(slug, false,
input);
53     [HttpPost("posts/{slug}/edit")]
54     public Task<IActionResult> SavePost([FromRoute] string slug, ContentPage input) => Save(slug, true,
input);
55     [HttpPost("posts/new")]
56     public Task<IActionResult> Create(ContentPage input) => Save(null, true, input);
57     private async Task<IActionResult> Save(string? oldSlug, bool detail, ContentPage input)
58     {
59         input.IsDetail = detail;
60         var page = oldSlug == null ? new ContentPage { IsDetail=true } : await
db.Pages.SingleOrDefaultAsync(p => p.Slug == oldSlug && p.IsDetail == detail);
61         if (page == null) return NotFound();
62         input.Id = page.Id;
63         if (!detail && oldSlug != input.Slug) ModelState.AddModelError("Slug", "Sabit sayfaların route'u
değiştirilemez.");
64         if (await db.Pages.AnyAsync(p => p.IsDetail == detail && p.Slug == input.Slug && p.Id != page.Id))
ModelState.AddModelError("Slug", "Bu slug zaten kullanılıyor.");
65         if (!ModelState.IsValid) return View("Edit", input);
66         var oldUrl = page.Url;
67         page.Title=input.Title; page.Slug=input.Slug; page.Description=input.Description ?? "";
68         if (detail)
69         {
70             page.CoverImageUrl = string.IsNullOrEmpty(input.CoverImageUrl) ? null :
input.CoverImageUrl.Trim();
71             page.Category = string.IsNullOrEmpty(input.Category) ? null : input.Category.Trim();
72             page.Author = string.IsNullOrEmpty(input.Author) ? null : input.Author.Trim();

```

Kod referansı · 73–89

Mevcut AdminController.cs; uzun satırlar yalnızca sayfaya sığmak için sarılır.

```
73         page.PublishedAtUtc = DateTime.SpecifyKind(input.PublishedAtUtc, DateTimeKind.Utc);
74     }
75     page.BodyHtml=detail ? BlogMarkup.ArticleBody(input.BodyHtml)
76         : page.Slug is "home" or "blog" ? BlogMarkup.ListingTemplate(input.BodyHtml) : input.BodyHtml;
page.Published=input.Published; page.UpdatedUtc=DateTime.UtcNow;
77     if (oldSlug == null) db.Pages.Add(page);
78     else if (detail && oldSlug != page.Slug)
79     {
80         // Existing template links follow the new detail slug.
81         foreach (var linked in await db.Pages.Where(p => p.BodyHtml.Contains(oldUrl)).ToListAsync())
82             linked.BodyHtml = linked.BodyHtml.Replace("\"" + oldUrl + "\"", "\"" + page.Url +
83             "\"").Replace("'" + oldUrl + "'", "'" + page.Url + "'");
84     }
85     try { await db.SaveChangesAsync(); }
86     catch (DbUpdateException) { ModelState.AddModelError("", "Kayıt tamamlanamadı. Slug başka bir
87     kayıta kullanılıyor olabilir."); return View("Edit", input); }
88     TempData["Success"] = "İçerik kaydedildi.";
89     return Redirect(detail ? $"/admin/posts/{page.Slug}/edit" : $"/admin/pages/{page.Slug}/edit");
90 }
```